

Full Stack Development

Project Name: TicTacToang Game

Group Number: 9

Members:

Lai Ho Thanh Hai
Nguyen Trong Khoa
Tran Le Phi Long
Nguyen Dung Tri

Course Lecturer: Dr. Tri Huynh

Submission date: 22/05/2026

"I declare that in submitting all work for this assessment I have read, understood and agreed to the content and expectations of the [Assessment declaration](#)."

Table of Contents

1. Introduction	3
2. Project Description	3
Main Features & User-Facing Functions:	3
Supported Use Cases:	4
Design Constraints & Limitations:	4
Team Learnings:	4
3. Implementation Details	4
3.1. Architecture Overview	4
3.2. Frontend and Backend Implementation	7
3.3. UI/UX Design	8
3.4. Tools and Package Selection	10
3.5. Development Process	11
3.6. Feature Checklist	12
3.7. Known Issues or Limitations	14
4. Evaluation	16
Test for Correctness & Expected Behavior:	16
Performance Optimization:	16
Feedback Considered:	16
Usability, Responsiveness, & Accessibility:	17
5. Conclusion	17
Applying LCCI is an effective method because it:	17
More time opportunities:	17
Technical & Design Reconsiderations:	18
6. Appendix	19
• Sequence Diagrams:	19
Sequence Diagram of Premium Subscription	19
Sequence Diagram of User Authentication	21
Sequence Diagram of User Registration	22
3.1	22
Frontend:	22
Backend:	26
ERD	31

1. Introduction

This project report documents the design and implementation of TicTacToang for the COSC2769/2808 Full Stack Development course. Our application brings the classic game of Tic-Tac-Toe into the modern digital era as a high-performance web platform. It removes traditional physical boundaries by allowing solo strategists to challenge AI, local friends to share a screen, and online players across the globe to compete seamlessly. The system supports two main user roles: registered Players who participate in matches, and Administrators who manage accounts and game rooms.

The overall scope encompasses building a full-stack web application featuring secure registration, login, profile management, premium billing, and real-time online multiplayer synchronization using WebSockets. To handle this efficiently, we organized our application into bounded contexts using a Modular Monolith architecture for both our backend and frontend. Internally, these domain modules follow a structured N-Tier design that uses Data Transfer Objects (DTOs) to safely isolate the presentation interface, business logic, and data access layers.

The primary objective of this report is to comprehensively outline our development methodology, system architecture, and UX/UI design choices to fulfill the course requirements.

2. Project Description

Our system is a complete web app with many features, including an extended 5-in-a-row version of tic-tac-toe. Built from a requirements-driven approach, it will include secure user authentication, complete user profile management, and various scalable gameplay modes, from local to real-time online matchmaking.

Main Features & User-Facing Functions:

- Robust Authentication: A secure authentication and login system with unique email registration, strong hashed passwords, and brute force protection. Access to backend APIs is restricted using stateless, secure JSON Web Signatures (JWS).
- Users can view a highly searchable, filterable history of past matches, allowing them to update contact details and automatically upload resized custom avatars.
- Dynamic Gameplay: Support of dynamic 10x10 & 15x15 grid; customizable board styles and markers. The game's win state is displayed as an animated sequence, and the match is tracked using King Chess algebraic notation.
- Single Player Mode with Three Difficulty Levels: a mode where players can fight against three different artificial intelligence levels. The Medium and Hard bot levels use pattern-recognition logic to actively resist player formations and seize winning opportunities (e.g., forks, 3-mark open lines).
- Synchronized Multiplayer Lobbies: Real-Time Online Arena, a WebSocket-based plugin that lets users create rooms and join matches, as well as chat in real-time.
- Premium Match Replay: As the name suggests, a system that will store every action of the matches and let premium users pause and play matches in order to repeat actions in any match.

Supported Use Cases:

- A new user registers, logs in, and sets up her profile using her own avatar.
- The player list shows two players on the same screen to play a local game, or a group of players to play a group of games.
- A hard AI that has the highest possible level of defense.
- Two players from different locations join through the online arena to play at the same time and communicate.
- A high-level player looking back on old tactics through the replay viewer.

Design Constraints & Limitations:

The system must conform to strict architectural constraints. The backend follows a strict N-Tier access pattern (Presentation -> Business Logic -> Repository), prevents data leakage through the use of Data Transfer Objects (DTOs), and is a Modular Monolith. Using a clean JavaScript layer, the frontend is completely componentized, with each UI component containing only its own JSX, event-handling Hooks, API calls, and CSS, resulting in loosely coupled components. API routes must go through a single REST HTTP Helper utility and be globally categorized.

Team Learnings:

Strict enforcement of architectural rules was learned. Dividing presentation logic from custom hooks and backend services, and abstracting them from vanilla JavaScript, led us to make the code more modular, cleaner, and very testable. We also learned how to use WebSockets for an online arena and the mathematical logic needed to build a light, responsive, and pattern-recognition AI without relying on massive external game engines.

3. Implementation Details

This is the core technical section. Describe how your system was constructed, which technologies and tools were used, and **justify your design choices**.

3.1. Architecture Overview

- Container (Architecture) Diagram:
 - [Frontend: Appendix](#)
 - [Backend: Appendix](#)
- Component Diagrams:
 - [Frontend: Appendix](#)
 - [Backend: Appendix](#)
- Entity-Relationship Diagram (ERD)

[ERD](#)

The visual ERD outlines the complete database schema. To avoid redundancy, standard CRUD attributes and basic foreign key linkages are omitted from this written analysis. The following points detail only the exceptional architectural and data-modeling decisions made for the TicTacToang platform.

Exceptional Entity & Attribute Designs

- **UserAccount and UserProfile Separation:**

Rather than embedding profile fields within a single account document, these are stored in separate MongoDB collections linked 1:1 by `_id`. This strict boundary isolates authentication concerns (password hashes, login attempts, lockout timers) from public display concerns (avatar, country, premium expiry). To prevent accidental data exposure, the password field utilizes Mongoose `select: false` and is never returned in queries unless explicitly requested.

- **AuthSession Token Hashing:**

The raw JSON Web Signature (JWS) is never persisted in the database. Only its SHA-256 hash (`hashSessionToken`) is stored. When a session is revoked upon logout, the raw token is added to an in-memory blacklist, providing immediate invalidation without requiring a database round-trip. The schema also utilizes a TTL (Time-To-Live) index on `expiresAt` for automatic cleanup.

- **GameSession Flat Array Board:**

The 10×10 or 15×15 grid is stored as `board: [String]`—a single-dimensional array of length 100 or 225. Cell access is computed dynamically via `idx = row * boardSize + col`. This deliberate choice avoids the overhead of nested arrays in MongoDB and simplifies the backend game engine, which operates exclusively on flat arrays.

- **Move Algebraic Notation:**

Every move stores its board position using the King Chess convention (e.g., columns as letters A-O, rows as numbers 1-15). Cell (0,0) accurately maps to "A1". This data is stored at write-time rather than computed at read-time to guarantee perfect replay accuracy for Premium users.

- **GameRoom Hybrid Entity:**

This entity serves dual roles: a REST-managed lobby listing and a WebSocket communication namespace. The `sessionId` field remains null initially and is only populated when a second player joins, meaning the room can exist independently before a game formally starts.

- **Dual-Purpose Transaction:**

A single collection handles both local wallet deposits and premium subscriptions. Stripe-initiated subscriptions create a "pending" transaction that an asynchronous webhook later marks as "completed," ensuring the transaction history remains the absolute source of truth for payment reconciliation.

- **AdminActionLog Polymorphic Metadata:**

The metadata field utilizes a Mongoose Mixed type to contain action-specific details (e.g., { previousStatus: true } for account deactivations). This prevents a sparse schema filled with nullable columns for disparate administrative action types.

Relationships Worth Noting

- **UserAccount 1 — 1 UserProfile:**

As noted above, these utilize the exact same `_id` but remain separated to enforce security boundaries and optimize distinct database access patterns.

- **GameRoom 1 — 0..1 GameSession:**

A room may exist without an active session (e.g., in a "waiting for opponent" state). The relationship is formalized only when the match begins.

- **UserAccount 1 — 0..N AuthSession:**

The system permits multiple active sessions per user to support multi-device logins. If an Admin deactivates an account, all associated sessions are revoked simultaneously via the `revokeAuthSessionsByUserId` service.

- [Sequence Diagrams:](#)

- Sequence Diagram of Premium Subscription

The financial architecture is broken down into four main processes which shown in Appendix:

- **Wallet Deposits:** Handles client-server requests for fetching current balances and processing direct fund deposits.
- **Internal Subscriptions:** Manages purchases using existing wallet funds, covering balance validation, database deductions, logging, and automated email confirmations.
- **Stripe Integration:** Generates pending checkout sessions and safely redirects the client to Stripe's external payment gateway.
- **Webhook Handling:** Processes Stripe's `checkout.session.completed` events to verify security signatures, finalize the transaction, extend premium access, and send the final receipt.

- Sequence Diagram of User Authentication

The complete end-to-end authentication lifecycle is broken down into four main processes which shown in Appendix:

- **Client-Side Validation:** The `useLogin` hook checks inputs directly in the `LoginForm`. If anything is missing or formatted incorrectly, it immediately throws an error without wasting a trip to the server.

- **Credential Verification:** The backend Auth Service queries the MongoDB User Collection to verify the password, returning a 401 Unauthorized error if the credentials don't match.
- **Token & State Management:** On a successful login, the server issues a JWT. The client saves this token in localStorage and updates the global AuthContext to keep the session active across the app.
- **Role-Based Redirection:** After fetching the user's profile with the new token, the router checks their permission level and redirects them to either the admin dashboard or the standard user interface.
- Sequence Diagram of User Registration

The registration flow is broken down into four main processes which shown in Appendix:

- **Client-Side Validation:** The RegistrationForm catches basic errors (like mismatched passwords) locally before sending anything to the server, keeping the backend load light.
- **Database Conflict Checks:** The backend queries the MongoDB database. If an email or username is already taken, it instantly aborts and sends back a 409 Conflict error.
- **Media Processing:** If a profile avatar is uploaded, the file is routed to Cloudinary for hosting. The system is set up to handle upload failures or grab the secure image URL on success.
- **Secure Provisioning:** After hashing the password for security, the final profile is saved to the database. The server issues a 201 Created response, triggering a success notification and redirecting the user straight to the login portal.

3.2. Frontend and Backend Implementation

The frontend is built with React and Vite, supported by React Router, Axios, Socket.IO Client, Bootstrap Icons, and canvas-confetti. The backend uses Node.js with Express.js and MongoDB through Mongoose. Additional libraries provide essential platform services: bcryptjs for password hashing, jsonwebtoken for authentication tokens, multer and Cloudinary for file uploads, socket.io for real-time communication, Stripe for payments, Nodemailer for email notifications, cors for cross-origin access, and dotenv for environment configuration. Together, these technologies provide a modern web stack capable of supporting secure login, live gameplay, payments, and media handling.

The backend API is organized by domain and exposed under route prefixes such as auth, users, game, multiplayer, billing, and admin. The authentication routes handle registration and login; registration accepts user details and an optional avatar, while login accepts an identifier and password and returns an access token with a sanitized user object. The user routes provide profile retrieval and updates, avatar uploads, and game history queries. The game routes support session creation, move submission, AI responses, session aborting, and replay retrieval for premium users. The multiplayer routes manage room creation, room listing, joining, closing, and room lookup. The billing routes support wallet balance retrieval, deposits, premium subscription purchase, Stripe checkout creation, webhook handling, and transaction history. The admin routes expose dashboard

metrics, player listings, and player status toggling, while some placeholder endpoints remain unimplemented.

In terms of data structure, the API consistently returns JSON objects with clear response shapes. Authentication responses include an access token and a user object containing fields such as id, username, email, role, premiumUntil, and avatar. Profile and user responses expose public account fields and avoid sensitive data. Game history items include session metadata, result, opponent, and participant summaries. Multiplayer room payloads include room number, player information, board configuration, status, session linkage, and timestamps. Billing responses include wallet balance, premium expiry, transaction records, and checkout URLs. This DTO-based approach improves consistency across endpoints and reduces unnecessary data exposure.

3.3. UI/UX Design

- Layout with responsiveness.
- Any system design or accessibility considerations.

Layout Strategy and Responsiveness

The front-end uses a structured, desktop-first layout that mixes CSS Grid and Flexbox. The strategy here is dividing up the heavy lifting between the two that keeps the browser rendering efficiently and stops elements from weirdly jumping around on the screen as the page loads.

CSS Grid is used exclusively for parts of the application that need a rigid, structured, two-dimensional layout. It handles the heavy lifting in three main areas:

- **The Navigation Bar (Navbar.jsx):** A strict three-column grid keeps the spacing perfectly uniform across the top of the application.

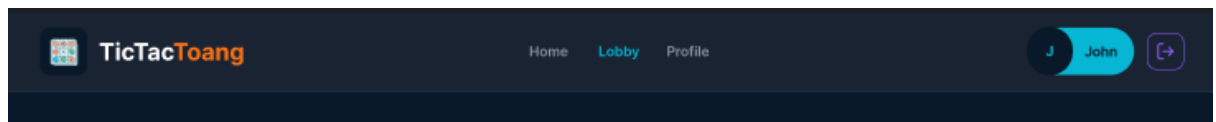


figure 1: Figma Navbar UI Design

- **The Multiplayer Arena (ArenaView.jsx):** A clean two-column grid splits the workspace, dividing the main gameplay surface from the interactive sidebar.

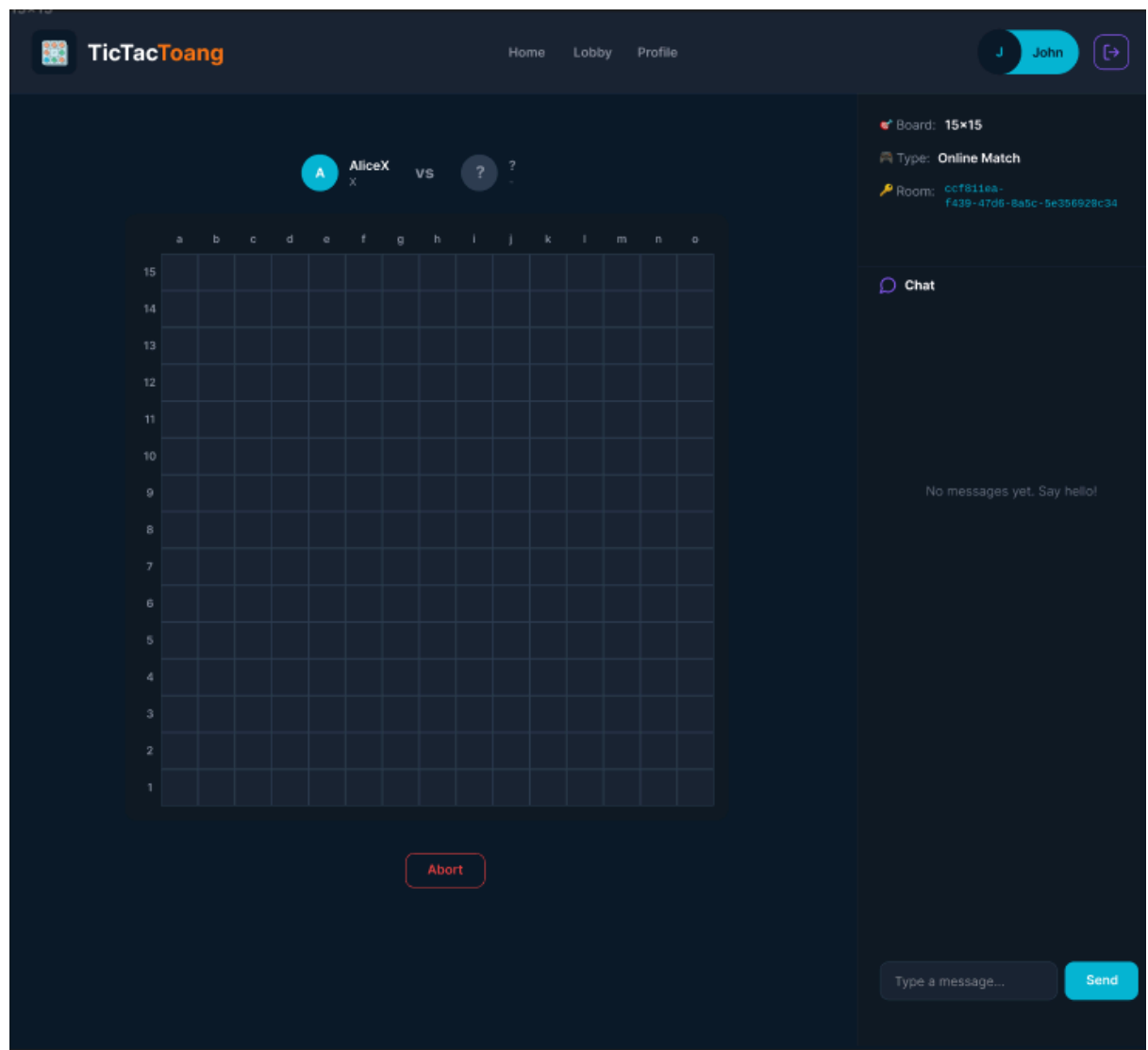


figure 2: Figma Arena View Design

- **The Game Board (GameBoard.jsx):** The core gameplay surface uses a dynamically generated grid that adapts directly to the chosen board dimensions. This ensures all the game cells are perfectly proportioned and evenly distributed, no matter the size of the board

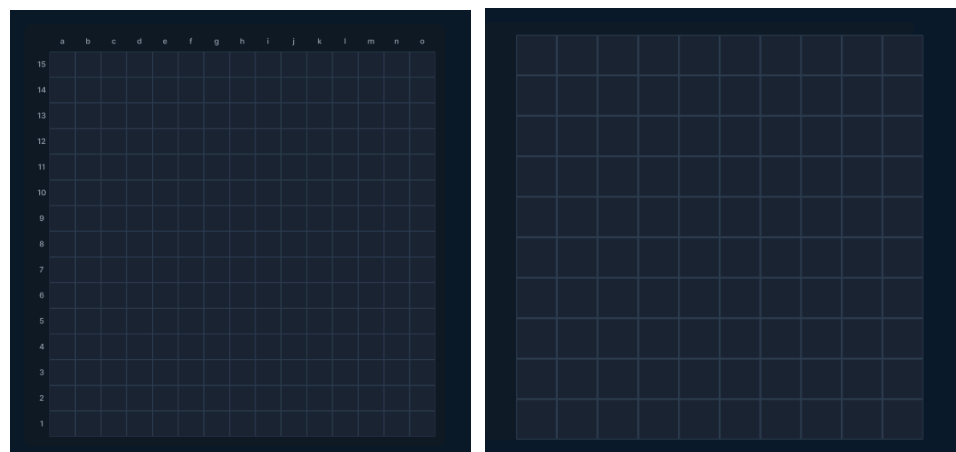


figure 3 Figma Game Board Design

System Design and Accessibility Considerations

The UI design focuses heavily on breaking things down. Layout logic, visual presentation, and data management are kept strictly separate to keep the codebase clean and organized.

- **Delegating the Rendering:** Complex screens are split into smaller, independent pieces. For example, GameBoard.jsx only worries about rendering the outer grid structure. The actual job of drawing and updating the individual squares is handed off completely to Cell.jsx.
- **Managing State (Encapsulation):** In the multiplayer lobby, RoomList.jsx has one main job: handling the network data (dealing with loading spinners, empty states, or error messages). Once the data successfully loads, it passes the information down to individual RoomCard.jsx components to actually display it on screen. This prevents unnecessary re-rendering and makes testing much easier.
- **Abstracting the Layout:** Global wrappers like AppLayout and Router.jsx are used to keep different parts of the application cleanly separated. This ensures that the regular public player interface stays totally isolated from the admin dashboards, completely preventing styles from accidentally bleeding over between different user views.

3.4. Tools and Package Selection

The project was built using a modern fullstack JavaScript architecture, with a [Node.js/Express](#) backend and a React frontend. Here are the tools and npm packages we used and why we chose them

- i) Core Frameworks and Database
 - Express: The foundation of our backend, used to handle all our routing and to build the RESTful API endpoints that our frontend communicates with
 - Mongoose and MongoDB: Chosen due to its flexible document structure which fits our data needs, ease of use, and our familiarity with it. We used mongoose to create strict schemas in our code so that user data and match history stay consistent
 - Database Hosting: We hosted our database on <http://localhost:3000> to make sure our backend could access the cloud data securely from anywhere
- ii) Real time features and network
 - [Socket.io](#): used to create a real time two way connection between the server and the players so that the game moves and updates sync instantly without needing to constantly refresh the page
 - Axios: installed on the frontend to handle our HTTP requests to the backend API. Makes fetching data and sending user input a lot cleaner than using the built in fetch API
 - Cors: used on the backend to avoid cross-origin errors, allowing our frontend domain to securely talk to our backend server
 - Dotenv: used to hide all of our sensitive information such as database passwords, JWT secrets, and API keys by keeping them in environment variables instead of hardcoding them
- iii) Security and Authentication

- JSONWebToken(JWT): used to handle user sessions. When a user logs in, the server generates a token so the frontend can keep them securely logged in and protect specific routes
 - Bcryptjs: since security is a priority, we used this to hash passwords before storing them in MongoDB. Even if the database is compromised, the actual passwords remain safe
 - React Icons: instead of downloading heavy images for UI icons, we used this package to easily import clean, lightweight icons directly into our components
- iv) File upload, emails, and payments
- Multer and Cloudinary: multer was used to handle incoming files on the backend, which are then uploaded directly to Cloudinary. Doing this keeps our main server lightweight since all images are stored externally in the cloud
 - Nodemailer: integrated this so our application can send automated emails directly from the server to users
 - Stripe: used to handle checkout workflows safely, meaning we can process payments without ever having to touch or store sensitive credit cards informations ourselves
- v) Deployment
- Frontend deployment: the frontend is hosted on render, automatically deploying whenever we push updates to our main branch
 - Backend deployment: the backend is hosted on render to keep our API alive and handle the open WebSocket connections for live gameplay

3.5. Development Process

The project timeline was broken down into short, one- to two-week development cycles called Sprints. To manage the workload, the GitHub Project Board was used. This setup kept everything organized and made it easy to see exactly what the team was working on at any given time.

Each Sprint followed a strict cycle:

At the beginning of each sprint, the team conducted a planning meeting to divide large features, such as the “Premium Subscription System,” into smaller and more manageable tasks. Each task was then recorded as a GitHub Issue and labeled with relevant categories such as frontend, backend, or database. After that, the tasks were allocated among the four team members according to their technical strengths and current workload to ensure balanced progress throughout the sprint.

During the sprint, tasks were tracked on a Kanban board through stages such as To Do, In Progress, In Review, and Done. The team mainly communicated asynchronously to share updates and resolve issues, while pair programming was used for more complex integrations between the backend and React frontend.

At the end of each sprint, the team reviewed the completed features. This process helped identify issues such as deployment firewall restrictions and environment variable misconfigurations, allowing the team to document the problems and improve the technical approach for the next sprint.

3.6. Feature Checklist

The following checklist showcases all the implemented features of our application, which were done accordingly to the project's functional and deployment requirements:

1. Player registration
 - Simplex:
 - Complete registration form: built a frontend form with mandatory fields for username, password, confirm password, and country
 - Unique email constraint: enforce a database level unique check to prevent duplicate accounts
 - Secure password hashing: implemented secure, one way password hashing using bcrypt before saving to the database
 - Country dropdown: configured a standardized selectable dropdown list for the country field
 - Medium:
 - Password strength validation: enforced criteria requiring at least 8 characters, 1 number, 1 special character, and 1 uppercase letter
 - Email syntax validation: implemented strict regex checking for standard email formatting(<255 characters, no spaces, correct domain layout)
 - Username validation: restricted usernames to english alphabet, numbers, underscores, and hyphens
 - Ultimo:
 - Dual layer validation and verbose error handling: duplicated all input validation on both the frontend and backend with clear error messages for instant user feedback and API security
2. Login and session management
 - Simplex:
 - Multi identifier authentication: allows users and admins to log in using either their username or email against hashed passwords
 - Medium:
 - Brute force attack protection: implemented a security block that locks an account after 5 failed login attempts within 60s
 - Identity propagation and session cleansing: frontend is securely informed of the user's role upon login, and all session data is wiped upon logout
 - Ultimo:
 - Stateless JWS authentication: generated JSON web signature (JWS) tokens via jsonwebtoken to securely authorize backend API requests
 - Token revocation and expiration: built a token blacklisting mechanism when a user explicitly logs out or when the token expires
3. Profile management
 - Simplex:
 - Profile editing: users can update their main contact details such as email, username, password, and country
 - Game session history: displays a log of past matches with timestamps, game types, and opponent name (including AI level if against bots)
 - Medium:

- Cloud based avatar optimization: handled profile picture uploads using multer and cloudinary, automatically resizes images for optimal performance
- Case insensitive pattern search: allows users to search their match history by match number or opponent names using partial, case insensitive strings
- Ultimo:
 - Advanced history filter and sorting: allows for full multi criteria filtering by date, result, game type, and chronological sorting in ascending or descending order
- 4. TicTacToe game mechanics
 - Simplex:
 - Core local gameplay: supports a 2 player local game on a single PC with customizable player turn orders and a standard 10x10 bordered grid. Features a 5 mark win condition, game abortion handling, and automated data recording
 - Medium:
 - Cosmetic and structural customization: allows players to choose from 3 gameboard styles, choose between 10x10 and 15x15 grids, and select from 6 different markers
 - Multi tiered AI opponent: offline single player mode has 3 levels of difficulties, being easy: simple moves, medium: basic blocking, hard: aggressive blocking and attacking patterns
 - Victory animations: added simple animation popup with motions and colors to announce the winner
 - Ultimo:
 - Real time web socket network arena: used socket.io for live, low latency online multiplayer matchmaking with an online game room lobby
 - Live match chat: added a real time text chat system during active online matches
 - Algebraic match replay system: added a coordinate system to record every move, allowing users to watch an interactive replay of past matches with pause, resume, forward, and backward controls
- 5. Premium subscription
 - Simplex:
 - Local wallet system: simulated a digital wallet where users can deposit funds and purchase a 10usd monthly subscription, triggering automated receipt emails with nodemailer
 - Medium:
 - Stripe payment gateway integration: upgraded the subscription model by embedding the official stripe SDK to handle secure, live 10 USD payments
- 6. Admin functionality
 - Simplex:
 - Global player directory: created an admin dashboard listing all the registered users, their premium status, and account balance
 - Medium:

- Administrative account interdiction: allows admins to deactivate and reactivate player accounts, instantly blocking deactivated users at the login endpoint
 - Ultimo:
 - Live room control and management: Allows admins to see all active live game rooms, search rooms by ID or player name, and forcefully terminate active rooms for server stability
7. Deployment status
- Simplex:
 - Local deployment environment: full system verified and running smoothly across containerized local development environments
 - Medium:
 - Global cloud deployment: deployed the fullstack architecture live on the cloud using render, making the app accessible globally

3.7. Known Issues or Limitations

1. Architectural & Technical Deviations This table outlines areas where the codebase currently deviates from the strict N-Tier and Modular Monolith specifications, as well as unresolved backend logic errors.

Category	File Location	Description & Impact
Defect	<code>multiplayerHandler.js</code>	Silently Failing Room Cleanup: The disconnect handler calls <code>multiplayerService.closeRoom()</code> , but the file imports the <code>multiplayerInterface</code> . This results in an undefined service call, leaving stale rooms active if the last player disconnects.
Defect	<code>admin.interface.js</code>	Data Transformation Error: The interface double-wraps the response using <code>adminDto.toPlayerResponse()</code> . Because the underlying service already returns data in a DTO shape, this second pass strips the <code>_id</code> field, returning <code>undefined</code> .

Architecture (A.3.1)	<code>multiplayer.controller.js</code>	Direct Service Coupling: The controller directly imports <code>userService</code> from the User module, bypassing the <code>userInterface</code> and violating strict module boundary encapsulation.
Architecture (A.3.2)	<code>admin.repository.js</code>	Layer Responsibility Breach: The Repository layer imports <code>adminDto</code> . The repository should remain completely unaware of Data Transfer Objects, handling only raw database models.
Implementation (A.3.a)	Frontend Components	Component Packaging Incomplete: Hooks currently reside at the module level rather than being strictly co-located within their owning component's package folder.

2. Functional & Feature Limitations This table details partially implemented features and UX/UI edge cases that do not severely break the core gameplay loop but impact the overall user experience and administrative control.

Feature ID	Limitation Description
6.3.1 - 6.3.3	Missing Admin Room Management: The backend lacks dedicated endpoints for Admin room management. As a workaround, the frontend currently calls standard multiplayer routes to interact with rooms.
6.3.3	Insufficient Authorization: The <code>POST /api/multiplayer/rooms/:id/close</code> endpoint is not properly gated for Admin roles. Any authenticated user currently has the technical capability to close an active room.

D.2.1	Outbound SMTP Blocked: Render's Free Tier blocks all outbound traffic on ports 25, 465, and 587. While the payment logic is robust, Premium Subscription confirmation emails silently fail to send in the production environment.
4.3.2 (Chat)	Volatile Chat History: In-game chat messages are not persisted to the database and are entirely lost upon page reload or socket disconnection.
UX/UI	Stale Lobby State: Selecting "Play Again" after a multiplayer match redirects both players back to the main lobby instead of instantly rematching them within the current active room.
UX/UI	Missing Wait States: Player 1 is presented with an empty, active board while waiting for Player 2 to join, lacking a clear "Waiting for opponent..." visual placeholder.
UX/UI	Volatile Customization: Custom board styles (Classic, Retro, Space) are maintained purely in the frontend state and are not persisted to the user's database profile, resetting upon reconnection.

4. Evaluation

Test for Correctness & Expected Behavior:

For all inputs, we used a two-layer validation approach: the first layer is a descriptive user feedback layer, and the second is a security validation layer on the backend. Specific unit tests were written for the AI logic to create possible board states (e.g., an open 4-mark line) and to ensure that the Medium and Hard bots exhibit the exact blocking behaviors required by the requirements.

Performance Optimization:

Considering our grid is 15x15, we took the liberty of optimizing our React rendering to avoid re-rendering the entire grid on each move. The JWS token implementation reduced lookups for user authorization in standard API calls, making a significant difference in the backend. We also ensured that if a user uploaded their logo to the image attachments section, the image is compressed and resized on the server so it loads quickly on the site's frontend.

Feedback Considered:

Peer testing revealed early on that there was difficulty in differentiating between AI difficulties. To overcome this limitation, we ensured the bots' names were recorded on the board

and had the Easy AI only grab cells adjacent to the player's last cell, making it easier to differentiate from the Hard AI, which was much more strategic.

Usability, Responsiveness, & Accessibility:

Responsive design principles are followed: The Application scales cleanly to mobile viewports, including the User Profile and all Admin UIs. High color contrast used on game board markers, no sole reliance on color to show the winner – motion animation and clear text announcements.

5. Conclusion

Applying LCCI is an effective method because it:

The key strength of our application is its extremely scalable and decoupled design. The codebase is well maintainable due to strict adherence to the Modular Monolith pattern on the back end and the 5-part componentization rule on the front end. Moreover, it adds WebSockets for live play and an amazingly powerful algorithmic AI, resulting in an extremely entertaining game experience.

More time opportunities:

The real-time chat would be further improved with the persistence of chat history and read receipts, which would be available over time. Further, we would update the algorithm to search more than one move ahead with an algorithm such as minimax/alpha-beta pruning, for an "Extreme" difficulty level.

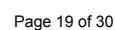
Technical & Design Reconsiderations:

If this were to be built again, we would definitely take a strong look at a global state management library, such as Redux. The more complex the Premium Replay feature became and the more complex maintaining the WebSocket state became, the more difficult it was to pass the state around locally between our separated hooks and JSX components. It would have been much easier to sync the network state to UI if there were a centralized Redux store.

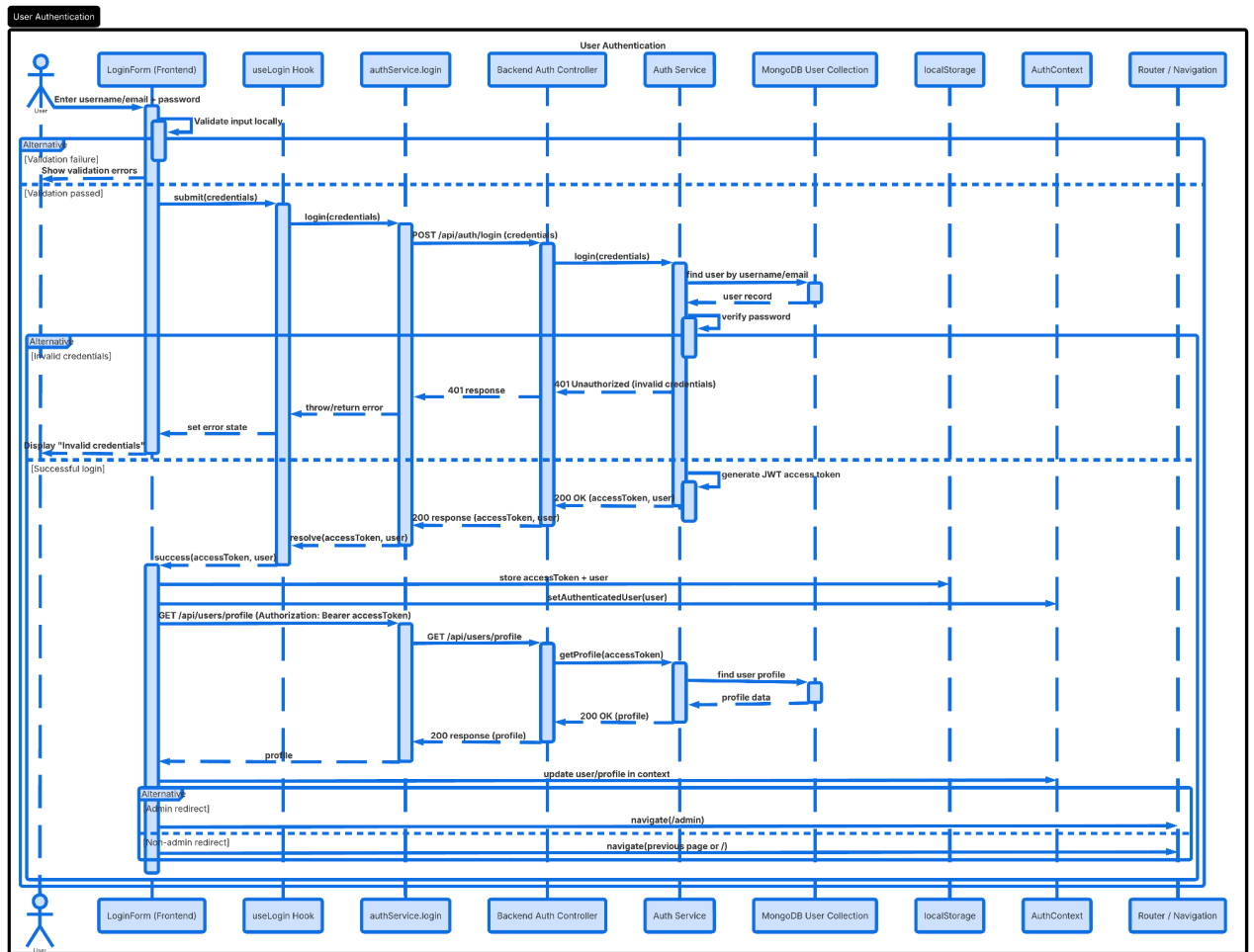
6. Appendix

- **Sequence Diagrams:**

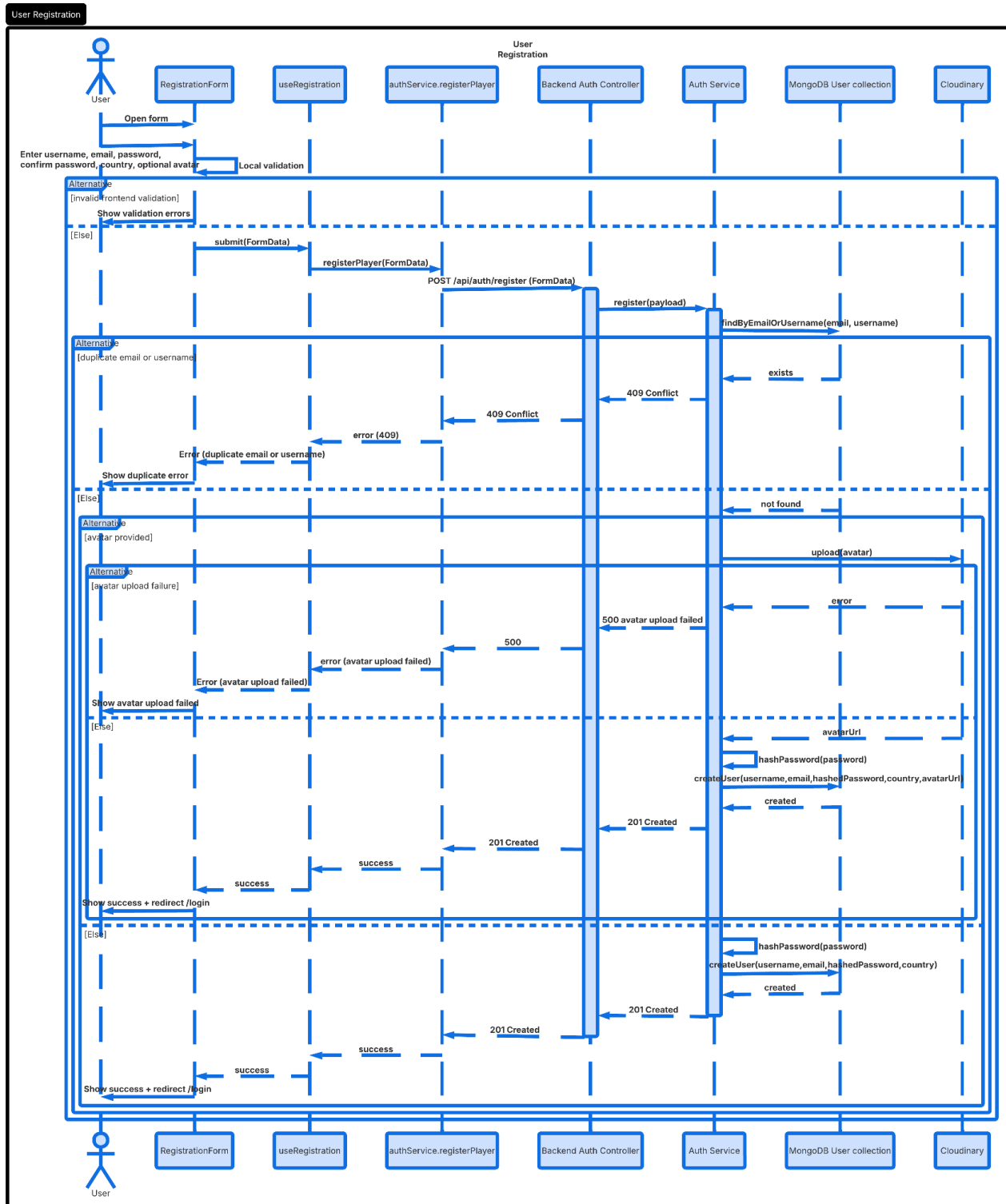
Sequence Diagram of Premium Subscription



Sequence Diagram of User Authentication



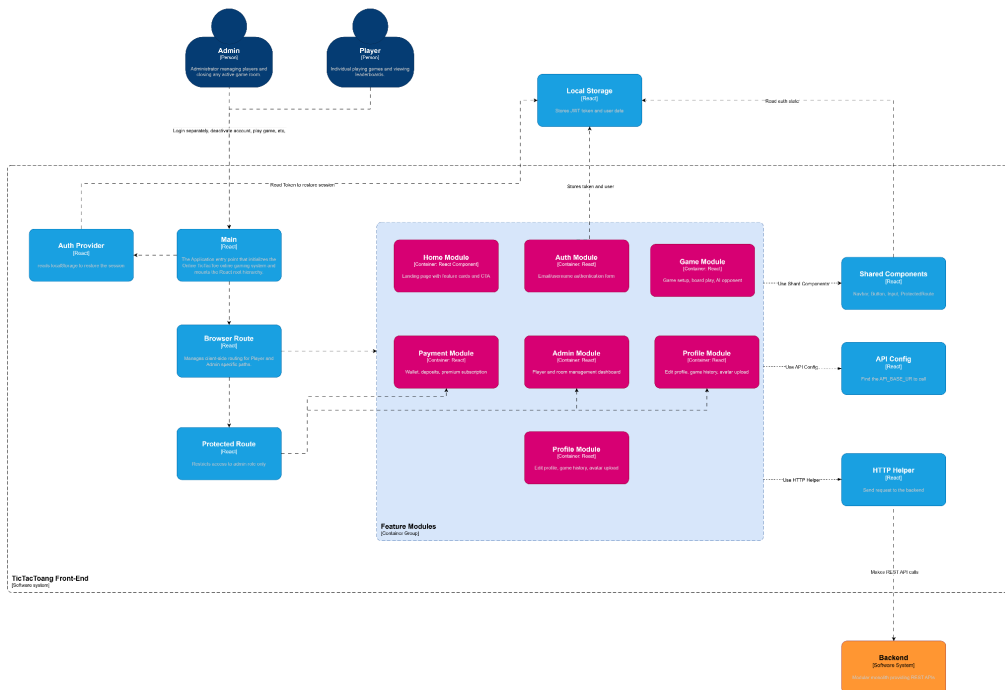
Sequence Diagram of User Registration



3.1

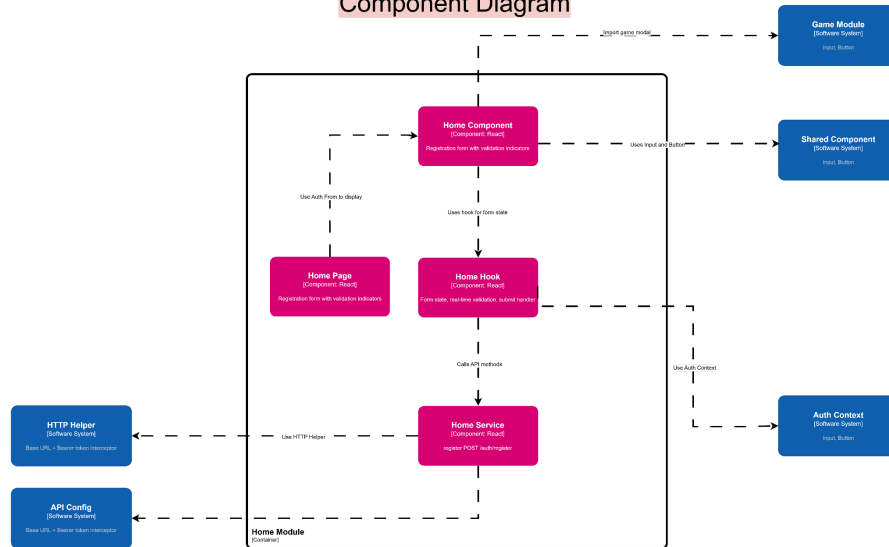
Frontend:
Container

Front-end Container Diagram

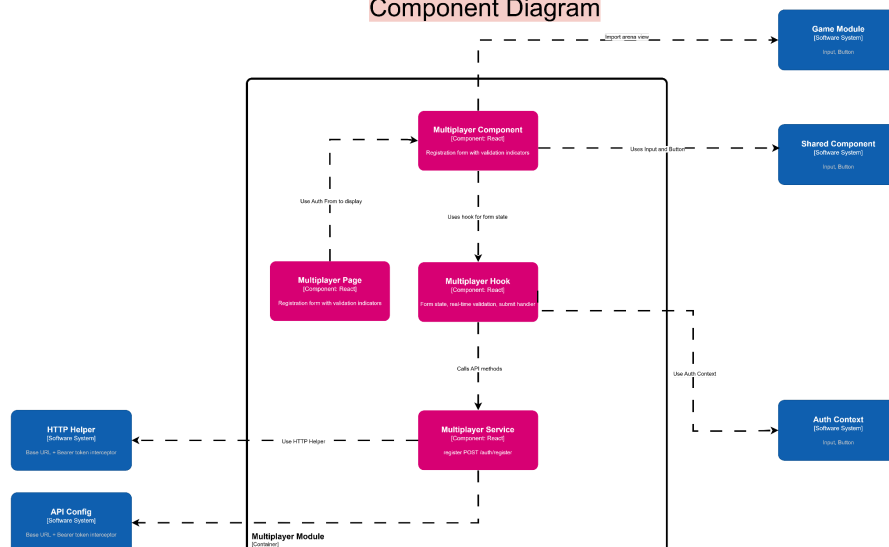


Component

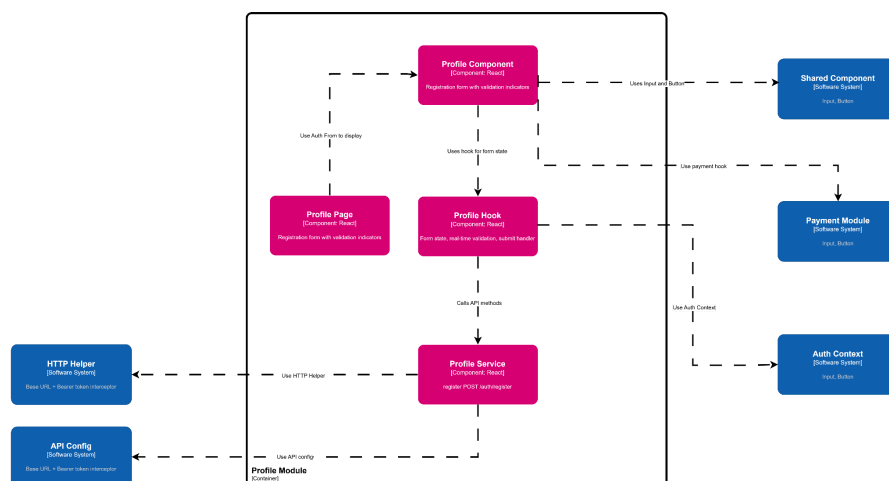
Home Module Front-end Component Diagram



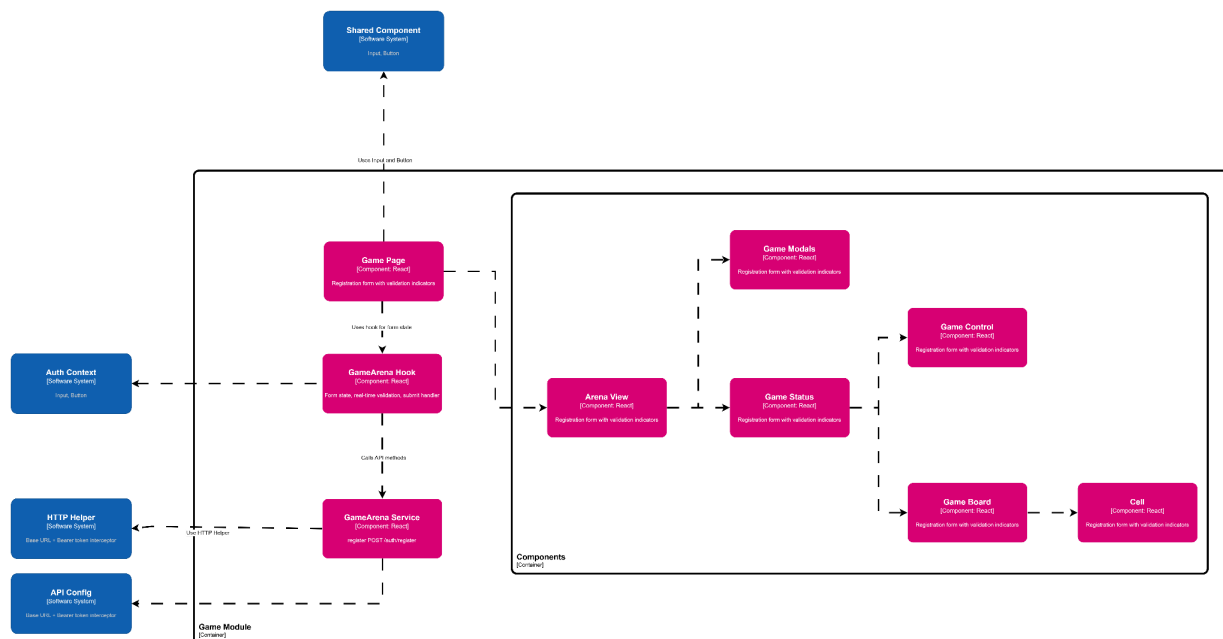
Multiplayer Module Front-end Component Diagram



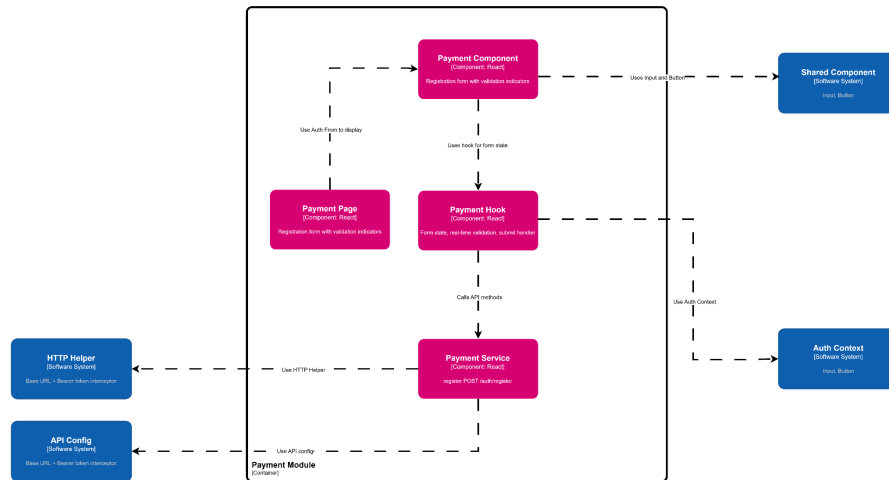
Profile Module Front-end Component Diagram



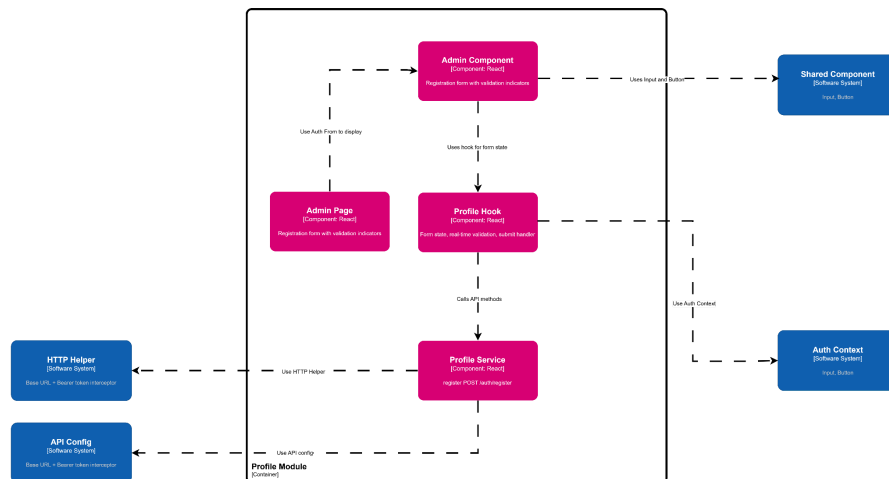
Game Module Front-end Component Diagram



Payment Module Front-end Component Diagram

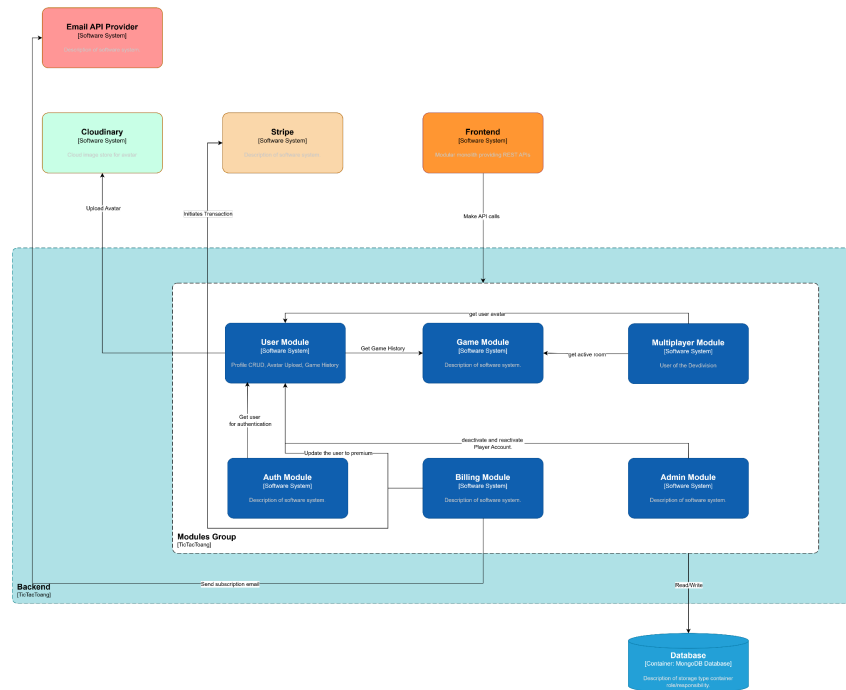


Admin Module Front-end Component Diagram



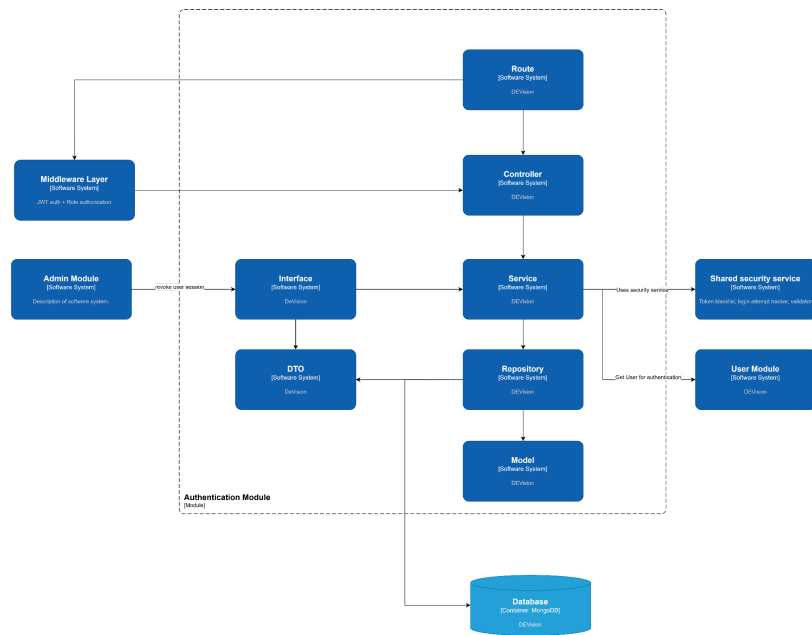
Backend:
Container

Back-end Container Diagram

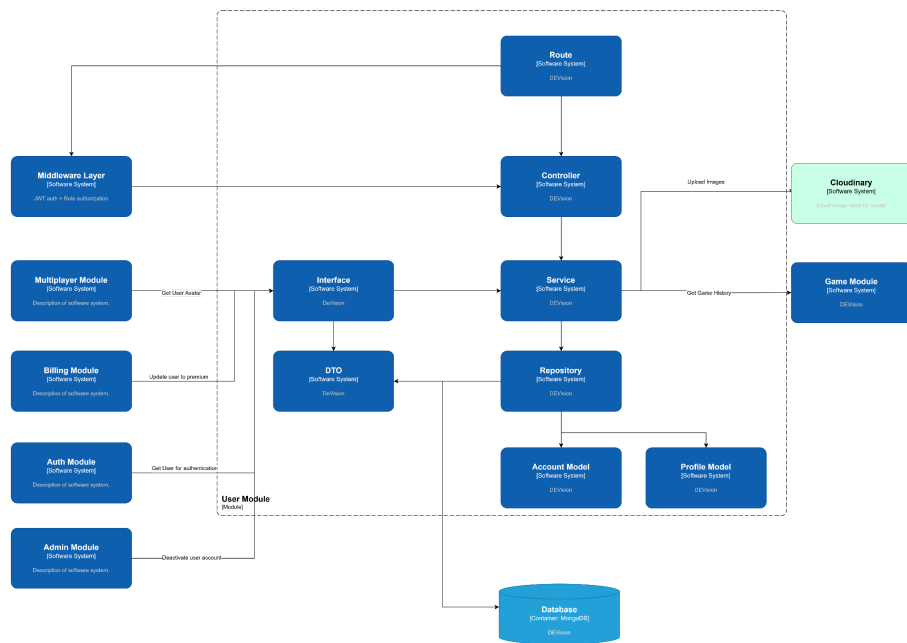


Component

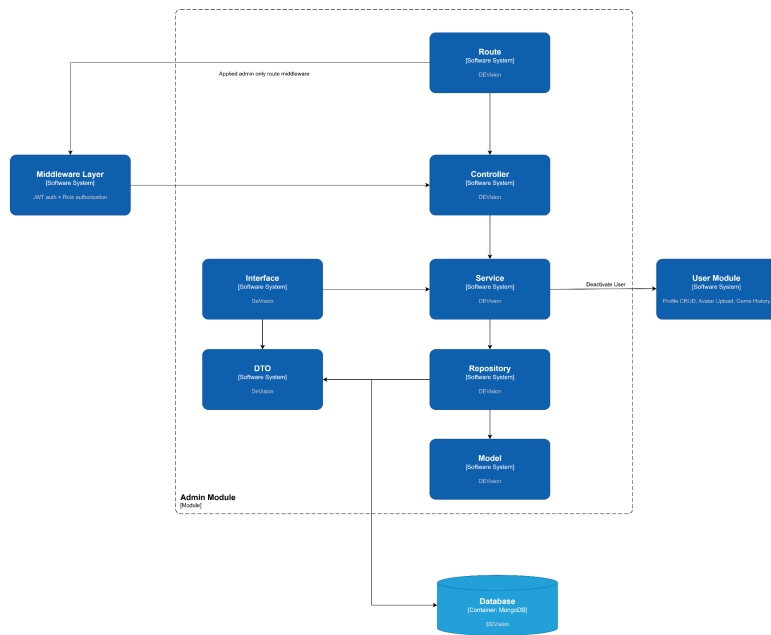
Authentication Modular Component Diagram



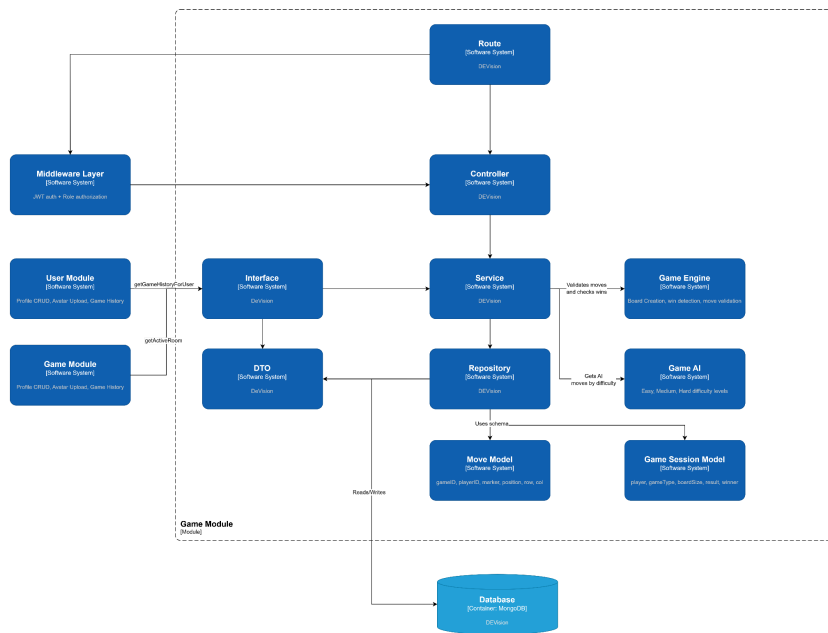
User Modular Component Diagram



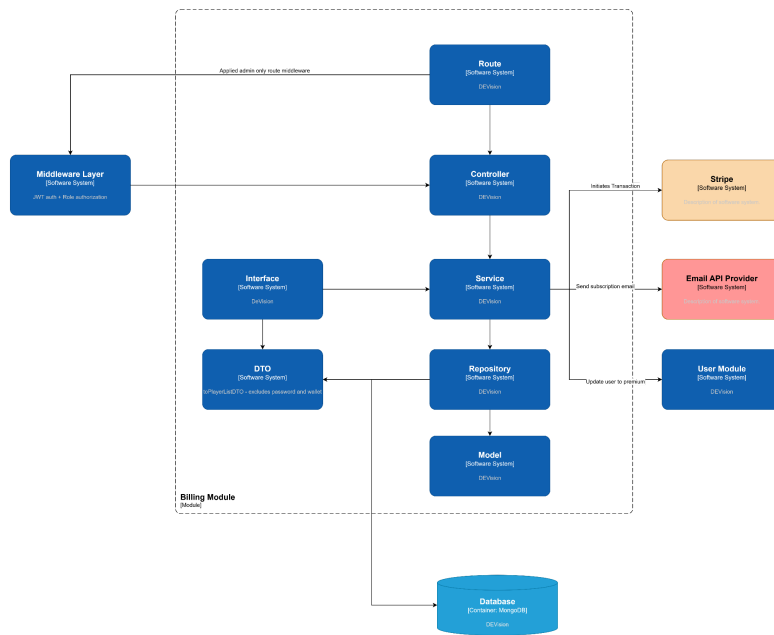
Admin Modular Component Diagram



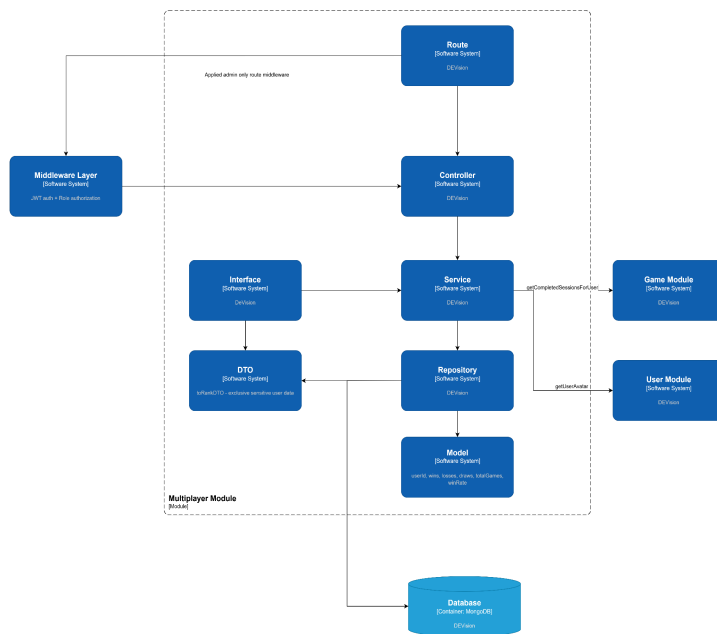
Game Modular Component Diagram



Billing Modular Component Diagram



Multiplayer Modular Component Diagram



ERD

